RDTII Rocky

Setup note: the installs, in order, for a computer that has nothing on it

Team: Rocky has a home run

6 October 2026

Before you start

Everything here starts from the terminal, the black window where commands are typed. Open it first and keep it open: every command below is pasted into it.

- **Windows**: press the Start button, type `PowerShell`, open *Windows PowerShell*.
- **Mac**: press `Cmd`+`Space`, type `Terminal`, press Enter.

This note installs what the tool needs, in order, and stops when the tool opens. You do not need to be a programmer: every command is copied and pasted whole. Allow 30 minutes and 2 GB of disk, plus 20 minutes and 9 GB for the free local AI engine (Section 5) and 2 GB for the meaning index (Section 6), both optional.

Where a program may already be on the computer, a *check* line comes first. If the window answers with a version number, skip the install and go on. If it says *not found* or *not recognized*, run the install lines under it. Whether everything works is not checked here: once the tool is open, the **Check** button on each stage page and the health dots along the top say what is missing.

When to open a new window. A window opened before a program was installed does not know that program exists. So after installing a *program* (Python, Tesseract, Ollama), close the window, open a new one and go back to the project folder as in Section 1. The *packages* installed with `pip` (Sections 3 and 6) need no new window: they go into the project's own folder, where every command finds them at once. The tool itself looks for packages only when it starts, so after Section 6 restart the tool, not the window. Each step below says which applies.

1 Get the project onto the computer

Download the project as a ZIP from the repository page on GitHub (green **Code** button → **Download ZIP**) and unzip it wherever you like to keep it; the Desktop is the example used below. GitHub names the unzipped folder after the branch or tag it came from, `rdtii-rocky-next-main` or `rdtii-rocky-next-final-submission`: rename it to `rdtii-rocky-next` (right-click the folder, *Rename*). It holds folders named `interface`, `stages` and `submission`.

Point the window at that folder; every later command assumes you are inside it. If the folder is not on your Desktop, change the path in the command to where you put it. The easy way: type `cd` and a space, drag the folder from its file window into the black window (its path appears there), then press Enter.

Windows (the folder on the Desktop):

```
cd $HOME\Desktop\rdtii-rocky-next
```

Mac (the folder on the Desktop):

```
cd ~/Desktop/rdtii-rocky-next
```

If `git` is already installed, `git clone https://github.com/JohnChen-kmg/rdtii-rocky-next` followed by `cd rdtii-rocky-next` and `git checkout final-submission` does the same, without the rename.

2 Python

The tool is written in Python, 3.12 or newer. Check:

Windows:

```
python --version
```

Mac:

```
python3 --version
```

If *not found*, if the Microsoft Store opens, or if the number is below 3.12, install: open https://www.python.org/downloads/ and download the latest version.

- **Windows**: in the installer, tick **"Add python.exe to PATH"** on the first screen before pressing Install; that tick is what makes the command above work.
- **Mac**: open the downloaded `.pkg` and accept every default.

Close the window, open a new one, go back to the project folder (Section 1) and check again.

3 The packages of the three stages

Four commands, run once, inside the project folder. The first gives the project its own package area (a folder called `.venv`, so nothing is installed into the computer at large). The second installs the packages of all three stages from one list; allow 5 minutes, it ends with `Successfully installed`. The third installs the extraction stage, which is part of the project itself. The fourth downloads the browser engine (about 150 MB) that the scraping stage uses on portals built with JavaScript. None of the four needs a new window afterwards.

Windows:

```
python -m venv .venv
.venv\Scripts\pip install -r requirements-demo.txt
.venv\Scripts\pip install -e stages/p2-extract
.venv\Scripts\python -m playwright install chromium
```

Mac:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements-demo.txt
.venv/bin/pip install -e stages/p2-extract
.venv/bin/python -m playwright install chromium
```

4 Tesseract — only for scanned documents

Lao PDR's laws are 96% scans, so Stage 2 reads them with OCR. Check:

```
tesseract --version
```

If *not found*, install:

Windows:

```
winget install UB-Mannheim.TesseractOCR
```

Mac (needs Homebrew; if `brew` is not found, install it from `https://brew.sh` first):

```
brew install tesseract
```

Close the window and open a new one, then check again. The language packs (English, Lao, Chinese, Portuguese) are already inside the project — nothing to download.

5 The free local engine (optional)

Mapping uses an AI engine. **Engine A** is hosted (Claude) and needs a key you pay for; you type it into the tool, on the Mapping page under **Engine API keys**, and it is held in memory only. **Engine B** runs on your own computer and is free. Everything else in the tool works without either: an engine is needed only to start a *new* mapping run. For Engine B, check:

```
ollama --version
```

If *not found*, install Ollama from https://ollama.com (a normal installer for Windows and Mac). Then close the window, open a new one, go back to the project folder and download the model the tool declares, about 9 GB:

```
ollama pull qwen2.5:14b
```

6 The meaning index (optional; needed for Chinese and Lao laws)

Before an engine reads anything, Mapping has to pick, out of thousands of provisions, the few worth reading for each indicator. The tool does this in two ways. The *keyword index* looks for the indicator's words; it is built in a minute and needs nothing beyond the packages of Section 3. The *meaning index* compares what a provision says with what an indicator asks, whatever words and whatever language are used, so it also finds the right provisions in Chinese or Lao laws, where keywords alone find almost nothing; the score thresholds, the tool's default selection rule, are read on it. It needs two more packages and, the first time a run builds it, a model download of about 2 GB; building it takes a while on a computer without a graphics card.

You can skip it for English laws: on the Mapping page set **Selection** to **Caps** and **Meaning index** to **Skip**, and the tool uses the keyword index alone. For Chinese or Lao laws, or to use the score thresholds, install it:

Windows:

```
.venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv\Scripts\pip install sentence-transformers
```

Mac:

```
.venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv/bin/pip install sentence-transformers
```

If the tool is already open, close it and start it again: it looks for packages only when it starts. The model download happens by itself during the first run that builds the index.

7 Start the tool

There is nothing to configure: no file to edit, no `.env`, no environment variable.

- **Windows**: double-click `Start-RDTII-Rocky.cmd` in the project folder.
- **Mac**: double-click `Start-RDTII-Rocky.command`.

The pages open in a window of their own, using the Edge or Chrome already on your machine; closing that window stops the tool.

A **Mac** may need one extra step, once: if the double-click opens the file as text, or says it cannot be executed, the unzip has dropped its permission to run. Give it back from the window, inside the project folder, then double-click again:

```
chmod +x Start-RDTII-Rocky.command
```

If the double-click does nothing, start it from the window instead and open http://127.0.0.1:8765/ in your browser; the message in the window says what is wrong.

Windows:

```
.venv\Scripts\python interface/app.py
```

Mac:

```
.venv/bin/python interface/app.py
```

From here the tool checks itself. The health dots along the top of the Overview show **Stages**, **Tesseract**, **Chromium**, **Ollama** and **Key** (grey for Ollama if you skipped Section 5, and for Key until you type one, which is correct), and the **Check** button on each stage page names anything still missing. Then carry on with the README's *Verify* section.

Questions: John (Jiaxiang) Chen, jiaxiangchen.kmg@gmail.com. The tool is Apache 2.0 licensed; this note covers the release tagged `final-submission`. The longer `SETUP_GUIDE.pdf` beside it explains what each package is for, what a working setup looks like and what to do when something is not found.